Rapport de Projet : FoSyMa

Groupe n° 14

Membre : Thomas LU 21128482

Lien vers le dépôt Git :
https://gitlab.com/thomaslu1245/dedale-etu-thomas-lu

À l'attention de :
Aurélie Beynier, Cédric Herpson, Vincent Corruble
LIP6 - Sorbonne Université

29 avril 2026

Table des matières

1 Introduction **2**

2 Architecture générale du code **2**
2.1 Abstraction et Spécialisation des Agents 2
2.2 Diagramme de Classes 2
2.3 Orchestration de la Chasse : Le Package `coalition_golem` 3
2.4 Dynamique des Comportements (*Behaviours*) 3
2.5 Modélisation de l'Environnement et Utilitaires 3

3 Choix de conception **3**
3.1 Stratégie d'Exploration Hiérarchique 3
3.1.1 Analyse Algorithmique 4
3.2 Protocoles de Communication et Partage de Connaissances 4
3.2.1 Protocole d'Échange Séquentiel (Ping-Pong-ACK) 5
3.2.2 Fusion des Connaissances et Cohérence 5
3.3 Coordination et Stratégie de Chasse 5
3.3.1 Localisation par traces olfactives (Protocole `smell`) 5
3.3.2 Infrastructure de la Coalition : `flood_optimisé` 6
3.3.3 Mécanique d'encerclement et Condition de Blocage 6
3.3.4 Dynamique des Automates (FSM) 6

4 Analyse des algorithmes **7**
4.1 Détails des métriques 7
4.2 Forces et limites du système 7
4.2.1 Forces 7
4.2.2 Limites et perspectives 8

5 Conclusion **8**
5.1 Synthèse 8
5.2 Regard critique et améliorations possibles 9

1 Introduction

Ce rapport présente le développement d'un système multi-agent capable d'explorer un environnement inconnu et de coordonner une chasse au golem...

2 Architecture générale du code

L'architecture de notre système a été pensée pour conjuguer robustesse opérationnelle et extensibilité logicielle. Elle s'articule autour d'un découpage strict entre la gestion des connaissances, la logique décisionnelle et les protocoles de communication.

2.1 Abstraction et Spécialisation des Agents

Au sommet de la hiérarchie se trouve la classe `AbstractCommunicationAgent`. Cette classe abstraite centralise l'intégralité des attributs nécessaires aux missions d'exploration et de traque.

Ce choix d'abstraction vise à garantir une extensibilité optimale : l'implémentation de futurs types d'agents s'en trouve grandement simplifiée. À titre d'illustration, j'ai développé la classe `ControlledAgent2`. Ce dernier, agissant comme une sentinelle délibérément statique, m'a servi de sujet d'expérimentation déterministe pour valider avec précision les protocoles de partage cartographique (*share-map*) et la bonne réception des directives opérationnelles, sans l'interférence de mouvements aléatoires.

2.2 Diagramme de Classes

Le diagramme suivant (Figure 1) détaille la structure de nos agents et de leurs comportements respectifs.

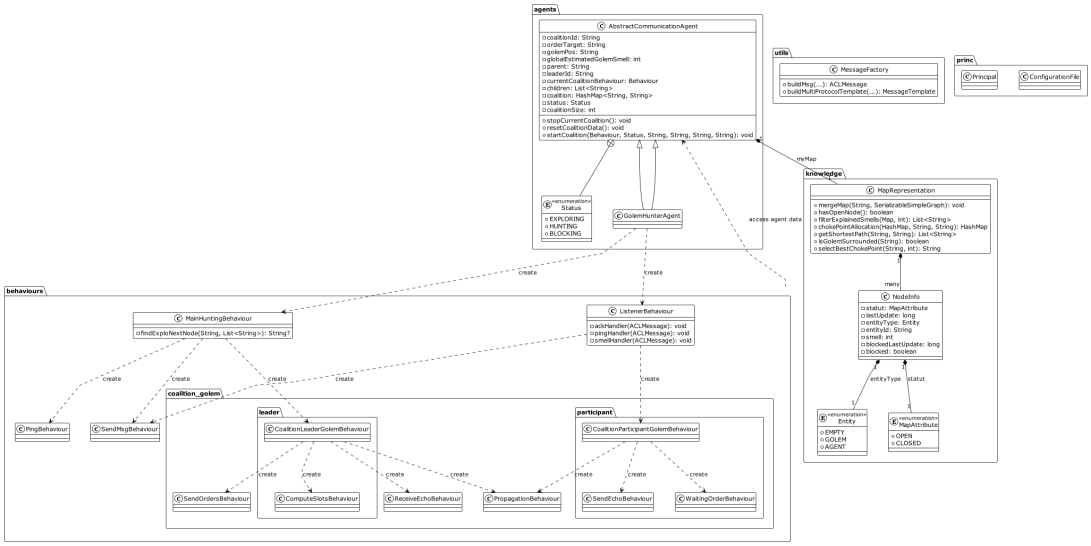


FIGURE 1 – Architecture modulaire Agents-Behaviours

2.3 Orchestration de la Chasse : Le Package `coalition_golem`

La logique de coordination, cœur névralgique de la traque, est confinée au sein du package `coalition_golem`. L'intelligence collective du groupe y est modélisée par un **Automate à États Finis (FSM)**, garantissant une transition fluide et rigoureuse entre les différentes phases de la mission. Ce package se structure comme suit :

- **Composants du Leader** : Deux classes dédiées à la prise de décision stratégique et à l'allocation des rôles.
- **Composants des Participants** : Trois classes régissant l'exécution des ordres et le suivi de proximité.
- **Socle Commun** : Trois classes transversales assurant la cohérence fonctionnelle entre les deux statuts.

2.4 Dynamique des Comportements (*Behaviours*)

Le cycle de vie de nos agents repose sur une dualité entre vigilance passive et action proactive :

- `ListenerBehaviour` : Ce comportement assure une veille informationnelle constante. Opérant en tâche de fond de manière asynchrone (sans déclencheur spécifique), il intercepte les messages imprévus. Il agit comme un catalyseur capable d'activer des déclencheurs critiques, tels que l'initialisation de l'automate de participation lors d'une sollicitation externe.
- `MainHuntingBehaviour` : Il constitue le cœur décisionnel de l'agent. Cette classe orchestre les routines principales de déplacement et l'analyse empirique de l'environnement immédiat par observation directe.
- **Abstraction des Échanges** : Afin d'épurer le code, les classes `PingBehaviour` (de type *OneShot*) et `SendMsgBehaviour` ont été implémentées. Elles encapsulent la complexité des protocoles de messagerie, évitant la redondance de blocs de code fastidieux et garantissant une normalisation des paramètres d'envoi.

2.5 Modélisation de l'Environnement et Utilitaires

La gestion topologique est confiée à la classe `MapRepresentation`. Véritable gestionnaire de la connaissance spatiale, elle assure l'incrémentation du graphe (`addNodes`), l'allocation stratégique des emplacements de chasse et le calcul de trajectoires via des algorithmes de **Dijkstra pondérés**.

Pour enrichir cette représentation, la classe `NodeInfo` a été introduite afin d'agréger une pluralité de descripteurs sémantiques par nœud. Enfin, la `MessageFactory`, application du patron de conception éponyme, centralise la genèse des messages et des masques de réception (*templates*), optimisant ainsi la célérité du développement et la maintenance du système de communication.

3 Choix de conception

3.1 Stratégie d'Exploration Hiérarchique

Le processus d'exploration de nos agents repose sur une prise de décision structurée en trois niveaux de priorité, garantissant une couverture optimale du graphe et une fluidité

de mouvement constante.

1. **Exploration Topologique (Priorité Haute)** : L'agent privilégie la résolution des "nœuds ouverts". Le choix de la destination s'appuie sur un algorithme de **Dijkstra pondéré**.
   — **Pondération dynamique** : Le coût d'une arête est modulé selon les entités présentes sur les nœuds.
   — **Gestion des obstructions** : Si un chemin calculé traverse une cellule marquée comme `blocked`, l'algorithme invalide le trajet (retour `null`), forçant l'agent à recalculer une alternative viable.
2. **Vigilance Temporelle (Priorité Moyenne)** : En l'absence de nouveaux nœuds à découvrir, l'agent bascule sur une stratégie de rafraîchissement. Il cible le nœud dont la dernière mise à jour (`lastUpdate`) excède 20 secondes le plus proche. Cette approche est cruciale pour localiser le Golem dans des zones déjà explorées mais potentiellement modifiées par ses déplacements.
3. **Comportement Stochastique (Priorité Basse)** : Si aucun des critères précédents n'est satisfait, l'agent effectue un mouvement aléatoire. Cette sécurité évite l'immobilisme de l'agent, prévenant ainsi l'obstruction involontaire des goulots d'étranglement du graphe pour les autres membres de la coalition.

3.1.1 Analyse Algorithmique

L'efficacité de la stratégie de déplacement repose sur la méthode `getShortestPath`, qui transcende le simple calcul topologique par l'intégration de contraintes sémantiques en temps réel.

— **Complexité temporelle** : Le calcul du chemin s'effectue en $O(V \log V + E)$, où V représente le nombre de nœuds du graphe connu et E le nombre d'arêtes. Cette complexité inclut :
   — Une phase de pré-traitement en $O(E)$ pour la mise à jour dynamique des poids des arêtes en fonction de l'occupation des nœuds par d'autres entités.
   — L'exécution de l'algorithme de Dijkstra exploitant ces poids pour identifier le trajet de moindre coût.
   — Une phase d'extraction et de filtrage du chemin en $O(V)$.
— **Optimalité et Robustesse** : L'algorithme est optimal vis-à-vis de la fonction de coût définie par la présence d'agents (coût de 10.0 pour les informations fraîches) ou d'obstacles. Contrairement à un parcours en largeur classique, l'implémentation assure une sécurité stratégique : si le poids total du chemin excède un seuil critique (1000.0), le chemin est considéré comme obstrué et la méthode retourne `null`. Cela permet de contourner intelligemment les zones de forte densité d'agents ou les impasses tactiques avant même d'engager le mouvement.

3.2 Protocoles de Communication et Partage de Connaissances

La cohérence de la vision globale de l'environnement repose sur un protocole d'échange de données cartographiques optimisé, visant à réduire la charge réseau tout en maximisant la fraîcheur des informations.

3.2.1 Protocole d'Échange Séquentiel (Ping-Pong-ACK)

Le partage de connaissances s'articule autour d'un cycle de vie en quatre étapes, déclenché périodiquement (tous les trois cycles d'exécution) ou par détection directe de voisins :

1. **Phase de Découverte (Ping)** : L'agent émet un message de *Ping* en mode *broadcast* ou de manière ciblée vers les voisins identifiés par observation directe. Ce message contient une estimation globale de l'odeur du Golem (`smellGlobalEstimation`). Bien que cette donnée soit liée à la chasse, son inclusion dans le *Ping* constitue une optimisation de "piggybacking", permettant de propager une information stratégique sans générer de messages supplémentaires.
2. **Synchronisation (Pong)** : À la réception, les agents disposent d'une fenêtre temporelle de 5 secondes pour manifester leur présence par un message de *Pong*.
3. **Transfert Différentiel (Share-Map)** : Afin d'éviter l'envoi redondant de la cartographie complète, l'émetteur utilise une structure de type `HashMap` pour mémoriser la date du dernier échange avec chaque collaborateur. La méthode `createSubGraph` est alors sollicitée pour générer un sous-graphe contenant exclusivement les nœuds dont la date de mise à jour (`lastUpdate`) est postérieure au dernier contact.
4. **Validation et Mise à Jour (ACK)** : Une fois le sous-graphe reçu et intégré via la méthode `mergeMap`, le destinataire émet un *ACK*. Ce signal final autorise l'émetteur initial à mettre à jour sa base de données temporelle pour les futurs échanges.

3.2.2 Fusion des Connaissances et Cohérence

L'intégration des données distantes par la méthode `mergeMap` n'est pas une simple concaténation, mais un processus de résolution de conflits basé sur deux critères :

— **Critère Temporel** : Une information n'est mise à jour que si l'horodatage entrant (`lastUpdate` ou `blockedLastUpdate`) est strictement supérieur à celui stocké localement.

— **Critère Hiérarchique** : Le statut des nœuds suit une règle de priorité interdisant, par exemple, la réouverture d'un nœud marqué comme fermé (`closed`) par une information obsolète ou moins prioritaire.

3.3 Coordination et Stratégie de Chasse

La phase de traque constitue l'aboutissement de la mission et s'appuie sur une transition fine entre la détection d'indices environnementaux et l'engagement d'une coalition coordonnée.

3.3.1 Localisation par traces olfactives (Protocole `smell`)

L'identification de la position du Golem repose initialement sur la détection de traces olfactives. L'odeur perçue par les capteurs ne fournit pas une coordonnée exacte mais une portée (*range*) permettant d'estimer la zone de présence du Golem.

— **Transition vers l'état *Smelling*** : Dès qu'une odeur est détectée, l'agent bascule du statut *exploring* vers le statut *smelling*. Un message de protocole `smell` est alors diffusé, contenant la liste des nœuds où des odeurs ont été identifiées.

— **Analyse et filtrage des odeurs** : Afin de distinguer un Golem libre d'un Golem déjà neutralisé, le système utilise la méthode `filterExplainedSmells`. Cette dernière compare les émanations observées avec les positions des Golems déjà encerclés dans un rayon donné (`radius`). Si le nombre d'odeurs détectées est supérieur au nombre de Golems identifiés comme bloqués, l'information est jugée suspecte (`suspiciousSmells`), déclenchant une investigation prioritaire.

3.3.2 Infrastructure de la Coalition : `flood_optimisé`

La détection directe du Golem déclenche l'initialisation du comportement `CoalitionLeaderGolemBeh` l'agent initiateur devenant alors le *Leader* de la traque. Le recrutement de la coalition s'appuie sur un mécanisme de **flood optimisé** :

— **Filtrage et Anti-spam** : Le `ListenerBehaviour` intercepte les messages `flood_golem`. Grâce à l'utilisation d'un `conversation_id`, tout message déjà traité est ignoré, ce qui stoppe immédiatement la propagation redondante et préserve la bande passante.

— **Gestion des conflits de leadership** : En présence de plusieurs leaders potentiels, un contrôle de délégation basé sur l'identifiant des agents est effectué. Un agent n'accepte de rejoindre une nouvelle coalition que si le leader émetteur possède une priorité supérieure, garantissant ainsi la stabilité des ordres.

3.3.3 Mécanique d'encerclement et Condition de Blocage

La stratégie de capture vise l'immobilisation totale de l'entité cible par l'occupation de son voisinage immédiat.

— **Allocation des cibles (*Slots*)** : Le leader définit la position du Golem comme le point de blocage central (*chokepoint*). Il alloue ensuite les nœuds adjacents à ses équipiers en fonction de leur proximité respective.

— **Prédiction de blocage par Dijkstra pondéré** : L'utilisation du **Dijkstra pondéré** joue un rôle crucial dans l'encerclement. En augmentant dynamiquement le coût des nœuds occupés par des alliés, l'algorithme incite les participants à converger vers le Golem par des chemins distincts. Ce comportement génère un mouvement d'étau naturel, fermant les issues du Golem de manière simultanée.

— **Validation du blocage** : Le succès de la capture est confirmé lorsque chaque nœud voisin de la position du Golem est soit occupé par une entitée, soit marqué comme `blocked` par la topologie de l'environnement.

3.3.4 Dynamique des Automates (FSM)

La synchronisation de la manœuvre est assurée par deux automates à états finis :

— **Le Leader (`CoalitionLeaderGolemBehaviour`)** : Il orchestre la diffusion du signal, la collecte des réponses (*échos*), le calcul des affectations de nœuds et l'envoi des ordres finaux de positionnement.

— **Les Participants (`CoalitionParticipantGolemBehaviour`)** : Ils assurent le relais du signal de flood, remontent les disponibilités de leur branche de l'arbre de propagation et exécutent l'ordre de mouvement dès réception de la cible assignée.

4 Analyse des algorithmes

Cette section détaille les performances théoriques des routines implémentées. Les métriques utilisées sont la complexité **T** (Temporelle/Calcul local) et la complexité **C** (Communicationnelle/Échanges réseau).

Algorithme	**Complexité (T/C)**	**Optimalité**	**Critère d'arrêt**
Exploration	T : $O(V \log V + E)$ C : $O(N)$ par cycle	Oui (complet)	Absence de `open_nodes` et rafraîchissement terminé
Chasse	T : $O(N_{agents} \times$ Dijkstra) C : $O(N)$ (flood optimisé)	Heuristique	Golem encerclé (voisins `blocked` ou agents)
Communication	T : $O(E)$ (prétraitement) C : $O(V_{diff} + E_{diff})$	Optimale (différentiel)	Réception du message de validation `ACK`

TABLE 1 – Analyse synthétique des algorithmes implémentés

4.1 Détails des métriques

— **Exploration** : La complexité temporelle est dictée par l'algorithme de Dijkstra pondéré ($O(V \log V + E)$) utilisé pour cibler les nœuds ouverts. L'arrêt est validé quand plus aucun nœud n'est accessible ou non visité.
— **Chasse** : Le leader effectue une allocation de cibles vers les nœuds adjacents au Golem. Le critère de succès est l'immobilisation totale de l'entité par une combinaison d'agents et d'obstacles naturels.
— **Communication** : Le protocole minimise la charge réseau en ne transmettant que les modifications du graphe (*subgraph*) depuis le dernier échange enregistré.

4.2 Forces et limites du système

L'évaluation du système met en évidence un équilibre entre l'efficacité des protocoles de communication et certaines contraintes topologiques liées à l'occupation de l'espace.

4.2.1 Forces

Le système se distingue par sa gestion optimisée des ressources et sa robustesse structurelle :

— **Efficience Communicationnelle** : L'utilisation de la méthode `createSubGraph` et du transfert différentiel réduit drastiquement la bande passante consommée. Le réseau n'est jamais saturé par des envois redondants.
— **Passage à l'Échelle (Scalability)** : Le mécanisme de `flood_optimisé`, incluant le filtrage par `conversation_id`, permet de coordonner un grand nombre d'agents sans explosion du trafic réseau.

— **Intelligence de Trajectoire** : L'intégration du **Dijkstra pondéré** permet de résoudre de manière élégante le problème de l'évitement de collisions et de l'encerclement. Le comportement d'étau est obtenu de façon émergente sans calcul géométrique complexe.
— **Résilience et Autonomie Décentralisée** : Le système fait preuve d'une grande robustesse face à la disparition imprévue d'unités. L'autonomie individuelle des agents garantit qu'aucune défaillance n'entraîne de blocage critique du reste de la flotte. Que la perte d'un agent survienne durant un échange cartographique, lors d'une coalition ou pendant un blocage, les agents restants poursuivent leurs objectifs sans interruption. Cette résilience s'applique également au leader : sa disparition n'est pas fatale à la mission.

4.2.2 Limites et perspectives

Malgré sa robustesse, l'approche actuelle présente des points de blocage potentiels :
— **Dilemme du "Gatekeeper" et Partitionnement** : Une limite majeure réside dans l'immobilité des agents lors d'une capture réussie. Si cet encerclement s'effectue dans un goulot d'étranglement, il peut partitionner le graphe en deux zones non connexes.
— **Unicité du golem dans l'interprétation** : Les agents voient qu'un seul golem sur leur carte actuellement, ce qui peut parfois entraîner une délégation d'une chasse presque terminée pour chasser un autre golem en passant à coté d'une autre coalition.

5 Conclusion

5.1 Synthèse

Ce projet a permis de concevoir et d'implémenter un système multi-agent décentralisé capable d'opérer avec efficience dans un environnement dynamique et initialement inconnu. La réussite de la mission de chasse repose sur la synergie de trois piliers technologiques :
— **Gestion Cartographique Intelligente** : L'utilisation de l'algorithme de **Dijkstra pondéré** a permis de fusionner la navigation, l'évitement de collisions et la stratégie d'encerclement en un mécanisme unique. En ajustant dynamiquement le coût des arêtes, les agents produisent un comportement d'étau de manière émergente.
— **Communication Sobre et Robuste** : Le protocole d'échange différentiel (Ping-Pong-ACK) via la méthode `createSubGraph` garantit la cohérence de la connaissance globale tout en minimisant la charge réseau par l'envoi exclusif des mises à jour temporelles.
— **Coordination Structurée** : L'implémentation d'automates à états finis (FSM) au sein du package `coalition_golem` a assuré une transition rigoureuse entre l'exploration individuelle et la traque collective, tout en gérant les conflits de leadership par délégation.

5.2 Regard critique et améliorations possibles

L'analyse post-développement révèle des axes d'optimisation pour accroître la performance globale de la flotte :

- **Résolution des Goulots d'Étranglement** : Le problème de partitionnement du graphe lors d'un blocage définitif constitue la limite principale du système actuel. Une amélioration majeure consisterait à implémenter un protocole de **cession de passage** ou de permutation circulaire des positions entre agents, permettant de maintenir la connectivité de la carte sans libérer la cible.
- **Arbitrage Multi-cibles** : La tendance actuelle à la focalisation sur un seul Golem pourrait être corrigée par un système de communication inter-leaders. Cela permettrait d'éviter qu'une coalition ne soit abandonnée au profit d'une autre chasse moins avancée, optimisant ainsi le temps de capture global.

En conclusion, bien que des défis subsistent concernant la gestion fine des ressources spatiales, l'architecture modulaire et les algorithmes de navigation implémentés constituent une base solide pour le déploiement d'une intelligence collective efficace.